SlowPaper: CLIP 다시 읽기

Learning Transferable Visual Models From Natural Language Supervision (Radford et al., OpenAI, 2021)
arXiv 2103.00020

수업에서 함께 본 CLIP 논문을 다시 읽고 다섯 가지 질문에 답한 회고입니다.

> 모두의연구소 AX Bootcamp · 컴퓨터 비전 · SlowPaper

1. 왜 이 논문을 선택했는가

수업에서 함께 읽은 논문이기도 하지만, 노드 8 실습을 하면서 이름만 여러 번 마주쳤기 때문이다.

- Stable Diffusion에 "a crack on the PCB" 같은 문장을 넣으면, 그 문장을 숫자로 바꾸는 텍스트 인코더가 CLIP이었다.
- AnomalyAny를 코랩에서 돌릴 때 OpenAI `clip` 패키지가 없어서 직접 설치했는데, 왜 결함 생성에 CLIP이 필요한지는 모르고 넘어갔다.

이미 쓰고 있던 부품이 무슨 원리로 돌아가는지 알고 싶어서 골랐다.

2. 어떤 연구 분야인가

사진과 글을 한 모델에서 같이 다루는 **멀티모달(Vision-Language)** 분야다. 사람이 붙인 클래스 번호 대신 인터넷 사진에 원래 달려 있던 글을 정답 삼아 배우고(**자연어 지도 학습, Natural Language Supervision**), 짝이 맞는 사진-글은 가깝게, 안 맞는 것은 멀게 두는 **대조 학습(Contrastive Learning)** 을 쓴다. 목표는 한 번 학습한 모델을 추가 학습 없이 새 분류 문제에 바로 쓰는 **제로샷 전이(Zero-shot transfer)** 다.

수업에서 이게 지도 학습인지 자기 지도 학습인지 헷갈렸는데, 사람이 라벨을 따로 달지 않고 데이터에 원래 붙어 있던 글을 정답으로 쓴다는 점에서 데이터 자체에서 정답을 얻는 쪽에 가깝다고 정리했다.

3. 어떤 문제가 있었는가

비싼 라벨링 비용과 고정된 클래스 목록.

기존 이미지 분류 모델은 `[cat, dog, car, …]`처럼 학습할 때 정한 목록 안에서만 답한다. 마지막 층의 출력 개수가 클래스 수로 고정되어 있어서, "고양이"를 "단모 고양이 / 회색 고양이"로 더 잘게 나누고 싶으면 그 사진을 모아 라벨을 다시 달고 다시 학습해야 한다.

공장으로 바꿔 보면, 불량을 "정상 / 긁힘 / 부품 누락"으로 학습한 분류기는 "납땜 과다"라는 새 불량이 나와도 셋 중 하나로 답한다. 새 불량이 생길 때마다 사진 수집, 라벨링, 재학습을 반복해야 한다.

방식	예	한계
사람이 직접 라벨링	ImageNet (클래스 1000개)	라벨링 비용이 커서 데이터를 키우기 어렵다
약한 라벨 대량 사용	인스타그램 해시태그, JFT-300M	데이터는 크지만 클래스 목록은 여전히 고정
글에서 직접 배우기	VirTex, ICMLM, ConVIRT	데이터가 작아 제로샷 성능이 낮다 (ImageNet 11.5%, Li et al. 2017)

4. 그 문제를 어떻게 해결하면 좋을까

① **라벨 대신 글을 정답으로**: 인터넷에서 사진-글 쌍 4억 개(WIT)를 모았다. 글은 사진에 원래 붙어 있던 것이라 라벨링 비용이 없다.

② **이미지와 글을 같은 공간에** (Figure 1):

- 이미지 인코더(ResNet 또는 Vision Transformer)는 사진을, 텍스트 인코더(Transformer, 글은 Byte Pair Encoding으로 쪼갬)는 글을 각각 임베딩 벡터(`e_img`, `e_txt`)로 바꾼다.
- 처음에는 두 벡터가 서로 다른 특징 공간에서 출발한다. 같은 대상(강아지 사진과 "강아지"라는 글)을 가리키면 두 벡터가 가까워지도록 학습해서, 결국 하나의 공간에 모이게 한다.

③ **대조 학습** (Figure 3 의사코드): 한 배치에 사진 N장, 글 N개를 넣고 모든 조합의 코사인 유사도(정규화한 벡터의 내적)로 N×N 표를 만든다.

```
          글1    글2    글3
사진1    [짝]    X      X
사진2     X    [짝]     X
사진3     X     X     [짝]
```

대각선(positive sample)은 유사도가 높아지고 나머지(negative sample)는 낮아지도록, 사진→글, 글→사진 두 방향으로 cross entropy를 구해 평균 낸다. 배치 크기가 32,768이라 한 번에 비교하는 negative가 아주 많고, 그래서 비슷한 것끼리도 구분하도록 배운다.

처음에는 사진을 보고 캡션을 단어 그대로 만들어 내게 했는데 너무 느렸다. "이 글 묶음 중 짝이 어느 것인가"만 고르게 바꾸면서 학습 효율이 크게 좋아졌다(Figure 2).

④ **제로샷 분류**: 클래스 이름으로 "a photo of a {클래스}." 문장을 만들어 텍스트 벡터로 바꾸고, 사진 벡터와 가장 유사한 문장의 클래스를 답으로 고른다. 클래스를 바꾸려면 문장만 바꾸면 되니, 3번의 고정된 목록 문제가 사라진다.

5. 시도한 결과는 어떠한가

결과	숫자
제로샷 ImageNet 정확도	11.5% (이전 연구) → **76.2%** (ImageNet 128만 장으로 학습한 ResNet-50과 비슷)
프롬프트 "A photo of a {label}."	클래스 이름만 쓸 때보다 +1.3%
프롬프트 80개 앙상블	추가 +3.5% (합쳐서 약 +5%)
제로샷 vs 라벨 몇 장 (Figure 6)	같은 특징으로 클래스당 4장을 학습한 선형 분류기와 평균적으로 비슷
분포 변화 (스케치, 그림 등, Figure 13)	ImageNet 대비 성능 하락 폭을 최대 75%까지 줄임

가장 기억에 남는 것은 **강건성(robustness)** 이다. 같은 바나나를 사진, 스케치, 그림으로 바꿔도 CLIP은 성능이 크게 떨어지지 않았다. 특정 데이터셋 하나에 맞춰 학습하지 않았기 때문에, 그 데이터셋에만 있는 우연한 패턴을 외우지 않은 것으로 이해했다.

한계 (논문 6장)

- 꽃·자동차 모델 같은 세밀한 구분, 물체 개수 세기 같은 추상적인 문제에 약하다.
- 학습 데이터에 거의 없던 종류에는 약하다. 손글씨 숫자 MNIST는 88%로, 픽셀에 바로 로지스틱 회귀를 쓴 것보다 낮다.
- 프롬프트를 어떻게 쓰느냐에 따라 결과가 달라지고, 각 분야 최고 성능에 닿으려면 연산이 약 1000배 더 필요하다고 추정한다.

6. 내 생각: 공장 결함 검사에 쓴다면

제로샷 그대로는 어려울 것 같다. 인터넷에 PCB 결함 사진과 그 결함을 설명한 글이 같이 올라온 경우는 드물어서, CLIP에게 PCB 결함은 MNIST처럼 처음 보는 데이터에 가깝다. 노드 8에서 Stable Diffusion이 PCB 균열을 잘 못 그린 것도, 텍스트 인코더인 CLIP이 "PCB 균열"을 제대로 알지 못했기 때문일 수 있다.

대신 CLIP의 이미지 특징은 그대로 두고, 결함 사진 몇 장으로 작은 분류기만 학습하는 방법(few-shot linear probe)은 해 볼 만하다. 논문에서도 제로샷이 클래스당 4장 학습과 비슷했으니, 라벨이 적은 공장에서 시작점으로 쓸 수 있을 것 같다. 결함 문장을 쓸 때는 "a photo of a printed circuit board with a missing component"처럼 물체와 결함을 구체적으로 적어야 한다.

회고

Keep

- 노드 8에서 이유를 모른 채 설치했던 `clip` 패키지와 SD 텍스트 인코더를 논문 구조와 연결해서 이해했다.
- 개념마다 공장 상황(새 불량 종류, PCB 결함 문장)에 대입해 봤다.

Problem

- 수업 때 듣기만 하고 바로 정리하지 않아서, 다시 읽을 때 처음 보는 것처럼 느껴진 부분이 많았다.

- 처음에는 두 인코더가 처음부터 같은 공간에 있다고 생각했다. 서로 다른 공간에서 출발해 학습으로 맞춰진다는 것을 다시 읽고 알았다.
- 지도 학습 / 자기 지도 학습 / 준지도 학습 중 어디에 속하는지 헷갈렸다.

Try

- ☐ 노드 6의 VisA pcb4 테스트 사진에 CLIP 제로샷("정상 PCB" vs "손상된 PCB")을 돌려 PaDiM의 AUROC와 비교해 본다.
- ☐ 같은 CLIP 특징으로 불량 라벨 1·4·16장 linear probe를 학습해, 몇 장부터 제로샷보다 좋아지는지 확인한다.
- ☐ 논문을 읽은 날 바로 다섯 가지 질문에 내 말로 답해 두는 습관을 들인다.